CSE 211 — Theory of Computation

Context-Free Grammars

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — CFG design and ambiguity in every paper
Lesson 18 — Grammars, Derivations and Parse Trees — 4-tuple, ⇒lm / ⇒rm, parse trees, yield, recursive inference, 'context-free'
Lesson 19 — Designing Context-Free Grammars — wrapping, concatenation, unions, counting, DFA → CFG
Lesson 20 — Ambiguity and Inherent Ambiguity — two parse trees, removing ambiguity (E/T/F), inherently ambiguous languages
Lesson 21 — The Pumping Lemma for Context-Free Languages — uvxyz, |vxy| ≤ p, standard proofs
Quick Notes — CFG — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lec07 - CFG.pdf (97 pages). Every grammar in the solutions was checked by a program that tests membership of all strings up to length 8 (6 for 3–4 letter alphabets) against the language definition; every ambiguity claim was checked by counting parse trees. CFG design and ambiguity questions are mapped to this note whichever section they were set in (only Hakim Sir's slides teach them); CNF/CYK is in Atif Sir's notes. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	CFG questions
2023-24	Q3(c) Why are CFGs called 'context free'? Examples (7) · Q5 CFG for {a^ib^jc^k | i = j or i = k; i,j,k ≥ 1}; show it is ambiguous (15)
2022-23	Q5 CFGs: balanced parentheses; at least two 1s (15) · Q6 CFG for {a^nb^nc^md^m} ∪ {a^nb^mc^md^n}; show it is ambiguous (15)
2021-22	Q1(e) Why are context-free grammars called 'context free'? (3) · Q2(d) CFG for {a^i b^j c^k | i ≠ j + k}, with design idea (15) · Q4(c) A B Sloppy's two CFGs for {a^ib^jc^k | i = j or j = k} — find the flaws (9) · Q4(d) Ambiguity of s and of the grammar in three parse-tree scenarios (10) · Q6 Ambiguous grammar / inherently ambiguous language; is S→AB|C, A→aAb|ab, B→cB|c, C→aCc|aDc, D→bD|b ambiguous? (15)
2020 (Jan term)	Q5 CFGs: at least one 0 and one 1; equal numbers of 0s and 1s (15) · Q6 CFG for {a^nb^nc^md^m} ∪ {a^nb^mc^md^n}; show it is ambiguous (15)
2019-20	Q5 CFGs: balanced parentheses; equal numbers of 0s and 1s (15) · Q6 Ambiguous grammar, inherently ambiguous language; is S→AB|C, A→aAb|ab, B→cB|c, C→aCc|aDc, D→bD|b ambiguous? (15)
2017-18	Q5 CFGs: equal numbers of 0s and 1s; at least three 1s (15) · Q6 E→E+E|E×E|(E)|a: two parse trees; remove the ambiguity (15)
2016-17	Q5(a) CFGs over {0,1}: odd length; at least three 1s; odd length with middle 0 (15) · Q8(b) CFG for {w ∈ {0,1}∗ | w = w^R and |w| divisible by 4} (10) · Q8(c) CFG that generates the given EXPR/TERM/FACTOR parse tree of a + a × a (10)
2015-16	Q4(a) G: R→XRX|S, S→aTb|bTa, T→XTX|X|ε, X→a|b — variables, terminals, strings in/not in L(G), description, T/F S1–S9 (7⅓) · Q4(b) CFGs: ≥3 1s; odd length with 0 in the middle; more 0s than 1s; {w#x | w^R substring of x} (10)
2014-15	Q5(a) Language of S→abScB|ε, B→bB|b (7) · Q5(b) CFGs over {0,1}: starts and ends with the same symbol; odd length with middle 0 (7) · Q6(a) Why S→0A|1B, A→0AA|1S|1, B→1BB|0S|0 is ambiguous (7)
2013-14	Q7(b) CFG for {a^i b^j c^k | i,j,k ≥ 0}; parse tree for ab²c³ (8⅓) · Q7(c) {a^n b^2n c^3n | n ≥ 0} is not context-free (6)
2012-13	Q5(a) Is E→E+E|E∗E|(E)|a ambiguous? Explain with a+a∗a (8) · Q6(a) L = {a^nb^nc^m} ∪ {a^nb^mc^m}: explain why all grammars for L are ambiguous (8) · Q7(a) CFG for 0∗1(0+1)∗; leftmost and rightmost derivations of 00101 (8)
2011-12	Q8(a) Formal definition of CFG; ambiguous grammar with example (6) · Q8(b) CFGs: {0^n1^n ∪ 1^n0^n}; strings with at least three 1s (10) · Q8(c) E→E+T|T, T→E∗F|F, F→(E)|a|b: derivation and parse tree of ((a+b)∗b) (7⅓)
2010-11	Q7 G: S→AA, A→AAA|a|bA|Ab — describe a derivation of b^m a b^n a b^p (10) · Q8 CFG generating all regular expressions over {a,b} (Σ = {a,b,(,),∪,∗,∅}) (10) · Q9 CFGs: 0^n1^m0^m1^n; a^nb^mc^k with n=m+k; with n=2m+3k; a^nb^m with n ≤ m ≤ 2n (10)
2009-10	Q6 Grammar <pop>→[<bop>,<pop>]|<bop>, <bop>→<boop>|(<pop>), <boop>→x|y|z: terminals/non-terminals, start symbol, parse tree for a sentence [sentence illegible] (10) · Q7 Grammar for palindromes over {0,1}; modify for alphabet {0,1,a} and even length only (10) · Q8 Recursive inference vs derivation for CFGs (10) · Q9 CFG for equal numbers of a's and b's; leftmost derivation of aabbab (10)
2008-09	Q8(a) L = {a^nb^nc^md^m} ∪ {a^nb^mc^md^n}: explain why all grammars for L are ambiguous (9) · Q8(b) CFG for 0∗1(0+1)∗; leftmost and rightmost derivations of 00101 (9) · Q8(c) S→aS|aSbS|ε: show aab has two parse trees, two leftmost and two rightmost derivations (5⅓)
2007-08	Q1(a) Properties of parse trees with examples (10) · Q3(b) Ambiguity in grammars; how to remove it; inherent ambiguity (8) · Q4(a) Grammar for palindromes over {0,1}; prove it correct by induction (10) · Q4(b) Recursive inference vs derivation for CFGs — differences and similarities (8)
2006-07	Q5 Define CFG; CFG for {a^m b^n | m > n, n ≥ 1} (10) · Q6 Parse tree, yield; declaration grammar D→TL; … : leftmost derivation of bin_ab00=101,b1,b1 and rightmost of bin_a=10,b=1 (10) · Q7 Show E→E+E|E∗E|(E)|I, I→a|b|Ia|Ib|I0|I1 is ambiguous; remove the ambiguity (10)

EXAM FOCUS Two fixed 15-mark questions in every recent paper: (1) 'Give CFGs for …' (balanced parentheses, equal numbers of 0s and 1s, at least k 1s — almost word for word in 2022-23, 2019-20, Jan-2020, 2017-18) and (2) 'Design a CFG for {a^nb^nc^md^m} ∪ {a^nb^mc^md^n} (or a^ib^jc^k with i = j or i = k) and show it is ambiguous' — 2023-24, 2022-23, 2021-22, 2019-20, Jan-2020. Section A adds short theory (why 'context-free', ambiguity scenarios) and harder designs (i ≠ j + k). Learn the design patterns of Lesson 19 and the ambiguity template of Lesson 20 first.

Lesson 18 — Grammars, Derivations and Parse Trees

Slides 1–29 (Lec07)

1. Why Grammars? (sl. 2–9)

A finite automaton asks 'is this string accepted?'; a grammar asks 'how could this string be built?' (generator view). Grammars suit recursive structure: nested brackets ([]{()}), nested calls encrypt(hash(msg)), mirrored packets 101#101. Balanced parentheses: S → SS | (S) | ε — ε is balanced, wrapping keeps balance, concatenation keeps balance. Derivation of (()()): S ⇒ (S) ⇒ (SS) ⇒ ((S)S) ⇒ (()S) ⇒ (()(S)) ⇒ (()()) (sl. 7–8). Finitely many rules, reused recursively, generate infinitely many strings (sl. 9).

2. Formal Definition (sl. 10–16)

$$G = (V, \Sigma, R, S)$$
$$\text{rules } A \to \alpha,\ A \in V,\ \alpha \in (V \cup \Sigma)^*$$

V: finite set of variables (non-terminals); Σ: terminals, disjoint from V; R: finite set of productions; S ∈ V: start variable (by convention the left side of the first rule). A sentential form is any string over V ∪ Σ appearing in a derivation; L(G) = {w ∈ Σ∗ | S ⇒∗ w}; L is context-free if L = L(G) for some CFG G (sl. 15–16).

EXAM-READY ANSWER Why 'context-free'? (2023-24 Q3(c), 2021-22 Q1(e)) Every production has a single variable on its left, A → α, so A may be replaced by α wherever it occurs, regardless of the symbols around it — the replacement does not depend on the context (sl. 14). Example: with S → 0S1 | #, the S in 00S11 is rewritten exactly as the S in 0S1. In a context-sensitive grammar a rule such as aAb → aγb may rewrite A only when it sits between a and b (Atif Lec 3 sl. 135); such grammars generate e.g. a^nb^nc^n, which no CFG can.

3. Leftmost and Rightmost Derivations (sl. 17–22)

Leftmost (⇒lm): always replace the leftmost variable; rightmost (⇒rm): the rightmost. They fix the order of expansion, not the structure. With E → E + T | T, T → T ∗ F | F, F → (E) | a:

```
leftmost:  E ⇒ E+T ⇒ T+T ⇒ F+T ⇒ a+T ⇒ a+T*F ⇒ a+F*F ⇒ a+a*F ⇒ a+a*a
rightmost: E ⇒ E+T ⇒ E+T*F ⇒ E+T*a ⇒ E+F*a ⇒ E+a*a ⇒ T+a*a ⇒ F+a*a ⇒ a+a*a
```

4. Parse Trees and Yield (sl. 23–28)

Conditions (sl. 24–25): (1) the root is the start variable; (2) every internal node is a variable; (3) every leaf is a terminal or ε; (4) if a node A has children X1 … Xk (left to right) then A → X1…Xk is a rule; a node using A → ε has the single child ε. The **yield** is the leaves read left to right, ignoring ε (sl. 26).

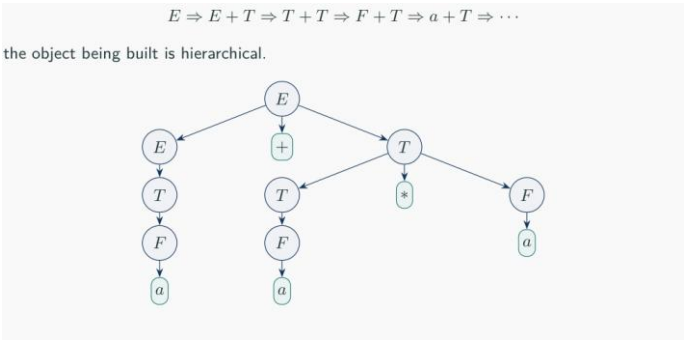


Figure 18.1 — Parse tree of a + a ∗ a in the E/T/F grammar (sl. 23)

MEMORIZE For a variable A and terminal string w the following are equivalent (sl. 28): A ⇒∗ w; A ⇒∗lm w; A ⇒∗rm w; there is a parse tree with root A and yield w. Each parse tree corresponds to exactly one leftmost and one rightmost derivation.

5. Recursive Inference (sl. 29)

Parse trees can be read top-down (derivation: start from S, expand variables until the string appears) or bottom-up (**recursive inference**: from the rule bodies whose symbols are already known to derive substrings, infer that the head variable derives their concatenation, until S derives w). CYK is a systematic bottom-up inference for CNF grammars.

	Recursive inference	Derivation
Direction	bottom-up: body → head	top-down: head → body
Starts from	terminal substrings of w	the start variable
Step	if A → X1…Xk and Xi derives wi, conclude A derives w1…wk	replace one variable in a sentential form by a body
Similarity	both prove w ∈ L(G); both correspond to the same parse tree	

Exam Approach

1. Variables/terminals/start: read off the rules (start = left side of the first rule unless stated).
2. Derivations: at each step underline the variable you replace; leftmost means always the first one.
3. Parse tree: draw from the root, one level per rule, then check that the leaves spell the string.

Practice — Lesson 18

(a) Actual — 2015-16 Q4(a). (b) Actual — 2014-15 Q5(a). (c) Actual — 2011-12 Q8(c). (d) Actual — 2016-17 Q8(c). (e) Actual — 2006-07 Q6. (f) Actual — 2010-11 Q7. (g) Actual — 2007-08 Q1(a), Q4(b) / 2009-10 Q8. (h) Actual — 2009-10 Q6.

(a) G: R → XRX | S, S → aTb | bTa, T → XTX | X | ε, X → a | b. (i) variables, (ii) terminals, (iii) start variable, (iv) three strings in L(G), (v) three strings not in L(G), (vi) describe L(G), (vii) True/False: S1 T ⇒ aba, S2 T ⇒∗ aba, S3 T ⇒ T, S4 T ⇒∗ T, S5 XXX ⇒∗ aba, S6 X ⇒∗ aba, S7 T ⇒∗ XX, S8 T ⇒∗ XXX, S9 S ⇒∗ ε. (7⅓)

(b) What language does S → abScB | ε, B → bB | b generate? (7)

(c) E → E + T | T, T → E ∗ F | F, F → (E) | a | b: derivation and parse tree of ((a + b) ∗ b). (7⅓)

(d) Give a CFG that generates the parse tree EXPR → EXPR + TERM, TERM → TERM × FACTOR, … for a + a × a. (10)

(e) D → TL; T → bin_ L → L, V | V V → I | I = N I → a | b | Ia | Ib | I0 | I1 N → 1 | N1 | N0. Derive bin_ab00 = 101, b1, b1; (leftmost) and bin_a = 10, b = 1; (rightmost). (10)

(f) G: S → AA, A → AAA | a | bA | Ab. For m, n, p > 0 describe a derivation of b^m a b^n a b^p. (10)

(g) Properties of parse trees; recursive inference vs derivation. (8–10)

(h) <pop> → [<bop>, <pop>] | <bop>, <bop> → <boop> | (<pop>), <boop> → x | y | z: terminals and non-terminals, start symbol and why, parse tree for the given sentence. (10)

✓ Solution

(a) (i) R, S, T, X. (ii) a, b. (iii) R. (iv) ab, ba, aab (also abab). (v) a, b, aba (also ε, bab). (vi) L(G) = all strings over {a, b} that are **not palindromes**: S puts different symbols at mirror positions, R wraps equal-length context around them (checked by program up to length 8). (vii) S1 F (one step gives XTX, X or ε),

S2 T, S3 F, S4 T (zero steps), S5 T, S6 F, S7 T (T ⇒ XTX ⇒ XX), S8 T (T ⇒ XTX ⇒ XXX), S9 F (S always produces a and b).

(b) S generates (ab)^n followed by n blocks cB in reverse order of generation; B = b^+. L = {(ab)^n (cb^(k1))(cb^(k2))…(cb^(kn)) | n ≥ 0, each ki ≥ 1} = {(ab)^n (cb^+)^n | n ≥ 0} (checked).

(c) Note T → E ∗ F (printed; the usual grammar has T → T ∗ F) — the derivation still exists and is unique:

```
E ⇒ T ⇒ F ⇒ (E) ⇒ (T) ⇒ (E∗F) ⇒ (T∗F) ⇒ (F∗F) ⇒ ((E)∗F) ⇒ ((E+T)∗F) ⇒ ((T+T)∗F) ⇒
((F+T)∗F)
  ⇒ ((a+T)∗F) ⇒ ((a+F)∗F) ⇒ ((a+b)∗F) ⇒ ((a+b)∗b)
Parse tree:  E−T−F−( E ) ; inner E−T−[E ∗ F] ; that E−T−F−( E ) ; E−[E + T], E−T−F−a, T−F−b ;
outer F−b
```

(d) EXPR → EXPR + TERM | TERM; TERM → TERM × FACTOR | FACTOR; FACTOR → a (optionally FACTOR → (EXPR)).

(e) Leftmost (always the first variable):

```
D ⇒ TL; ⇒ bin_L; ⇒ bin_L,V; ⇒ bin_L,V,V; ⇒ bin_V,V,V; ⇒ bin_I=N,V,V; ⇒ bin_I0=N,V,V;
  ⇒ bin_I00=N,V,V; ⇒ bin_Ib00=N,V,V; ⇒ bin_ab00=N,V,V; ⇒ bin_ab00=N1,V,V; ⇒ bin_ab00=N01,V,V;
  ⇒ bin_ab00=101,V,V; ⇒ bin_ab00=101,I,V; ⇒ bin_ab00=101,I1,V; ⇒ bin_ab00=101,b1,V;
  ⇒ bin_ab00=101,b1,I; ⇒ bin_ab00=101,b1,I1; ⇒ bin_ab00=101,b1,b1;
Rightmost (always the last variable):
D ⇒ TL; ⇒ TL,V; ⇒ TL,I=N; ⇒ TL,I=1; ⇒ TL,b=1; ⇒ TV,b=1; ⇒ TI=N,b=1; ⇒ TI=N0,b=1;
  ⇒ TI=10,b=1; ⇒ Ta=10,b=1; ⇒ bin_a=10,b=1;
```

Parse tree: D has children T, L, ';'; the yield of a parse tree is its leaves read left to right. Both strings have exactly one parse tree (checked).

(f) S ⇒ AA. Expand the first A: apply A → bA m times, then A → Ab n times, then A → a: A ⇒∗ b^m A ⇒∗ b^m A b^n ⇒ b^m a b^n. Expand the second A: apply A → Ab p times, then A → a: A ⇒∗ a b^p. Hence S ⇒∗ b^m a b^n a b^p.

(g) Properties: §4 conditions (root S, internal nodes variables, leaves terminals/ε, each node + children is a rule); yield = the derived string; one tree ↔ one leftmost ↔ one rightmost derivation; a string with two trees shows ambiguity. Recursive inference vs derivation: table in §5.

(h) Non-terminals <pop>, <bop>, <boop>; terminals [ ] , ( ) x y z. Start symbol <pop> — it is the left side of the first rule and the only variable not used solely inside another construct. [The sentence in 2009-10 Q6(c) is illegible.] Example tree for [x, (y)]: <pop> → [<bop>, <pop>]; <bop> → <boop> → x; <pop> → <bop> → (<pop>); <pop> → <bop> → <boop> → y.

Lesson 19 — Designing Context-Free Grammars

Slides 30–52 (Lec07)

1. Design Checklist (sl. 31) and Patterns (sl. 52)

Pattern	Rule form	Use
Wrapping	S → aSb	matched ends: a^nb^n, palindromes, w#w^R
Concatenation	S → SS or S → AB	independent parts, balanced strings
Optional structure	A → α | ε	zero or more
Union of cases	S → S1 | S2	'or' conditions, i = j or j = k
Regular part	one variable per DFA state	at least k 1s, substrings
Syntactic categories	one variable per category	expressions, statements, queries

Ask: smallest member? how is a bigger member built from smaller ones? natural first/last pair? a union of simpler languages? a regular part? why can no invalid string be generated?

2. The Slides' Worked Grammars (sl. 32–51)

```
mixed brackets      S → SS | (S) | [S] | {S} | ε
#0 = #1             S → 0S1S | 1S0S | ε        (split at the first point where counts are equal)
w#w^R               S → 0S0 | 1S1 | #
palindromes         P → 0P0 | 1P1 | 0 | 1 | ε
a^n b^m c^m d^n     S → aSd | T,  T → bTc | ε
i = j or j = k      S → S1 | S2; S1 → AC, A → aAb | ε, C → cC | ε; S2 → DB, D → aD | ε, B → bBc | ε
#0 ≠ #1             U → A | B; A → E0E | E0A; B → E1E | E1B; E → 0E1E | 1E0E | ε
DFA → CFG           R_i → a R_j for each δ(q_i, a) = q_j;  R_i → ε for accepting q_i  (regular ⊂ CFL)
trap: S → 0S0 | 1S1 | ε generates ww^R, NOT ww (sl. 51)
```

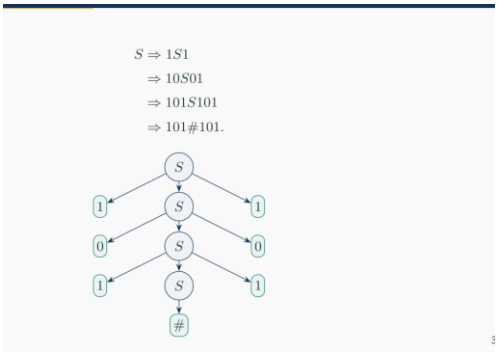


Figure 19.1 — Wrapping: derivation and tree of 101#101 (sl. 39)

EXAM-READY ANSWER Completeness of S → 0S1S | 1S0S | ε (sl. 37): a non-empty string w with #0 = #1 starting with 0: scan left to right until the prefix first has equal counts; that prefix is 0x1 with x balanced, and the rest y is balanced, so w = 0x1y and S → 0S1S generates it by induction. Symmetric for 1. Soundness: every rule adds one 0 and one 1.

Exam Approach — CFG Design

1. Write 3 members including the smallest; spot the matched pairs or the union.
2. Choose the pattern; introduce one variable per independent obligation (outer match, inner match, free part).
3. Write the rules; derive one member; argue in one line why nothing else is generated.

4. If the question says 'explain the design idea first', write the decomposition (e.g. a^k (a^j b^j) c^k) before the rules.

Practice — Lesson 19

(a) Actual — 2022-23 Q5, 2019-20 Q5, Jan-2020 Q5, 2017-18 Q5. (b) Actual — 2016-17 Q5(a), 2015-16 Q4(b), 2014-15 Q5(b), 2011-12 Q8(b). (c) Actual — 2021-22 Q2(d). (d) Actual — 2010-11 Q9. (e) Actual — 2006-07 Q5.

(a) Give CFGs over {0, 1}: balanced parentheses (over {(,)}); equal numbers of 0s and 1s; at least one 0 and at least one 1; at least two 1s; at least three 1s. (7 + 8 each)

(b) CFGs: odd length; odd length with middle symbol 0; starts and ends with the same symbol; more 0s than 1s; {w#x | w^R is a substring of x}; {0^n1^n} ∪ {1^n0^n}. (5–10 each)

(c) Design a CFG for {a^ib^jc^k | i ≠ j + k; i, j, k ≥ 0}. Explain your design idea first. (15)

(d) CFGs: 0^n1^m0^m1^n; a^nb^mc^k with n = m + k; with n = 2m + 3k; a^nb^m with n ≤ m ≤ 2n. (10)

(e) Define CFG; design a CFG for {a^mb^n | m > n, n ≥ 1}. (10)

✓ Solution

```
balanced parentheses      S → (S)S | ε                (unambiguous; S → SS | (S) | ε also works)
equal 0s and 1s           S → 0S1S | 1S0S | ε
≥ one 0 and ≥ one 1       S → X0X1X | X1X0X,   X → 0X | 1X | ε
≥ two 1s                  S → X1X1X,           X → 0X | 1X | ε
≥ three 1s                S → X1X1X1X,         X → 0X | 1X | ε
```

Derivation of 0110 in S → 0S1S | 1S0S | ε: S ⇒ 0S1S ⇒ 01S ⇒ 011S0S ⇒ 0110S ⇒ 0110.

```
odd length                S → X | XXS,  X → 0 | 1
odd, middle 0             S → 0S0 | 0S1 | 1S0 | 1S1 | 0
starts & ends same        S → 0X0 | 1X1 | 0 | 1,   X → 0X | 1X | ε
more 0s than 1s           A → E0E | E0A,   E → 0E1E | 1E0E | ε
w#x, w^R substring of x   S → TX,  T → 0T0 | 1T1 | #U,  U → 0U | 1U | ε,  X → 0X | 1X | ε
{0^n1^n} ∪ {1^n0^n}       S → A | B,  A → 0A1 | ε,  B → 1B0 | ε
```

w#x: write x = u w^R v. T generates w # u w^R (wrapping matches the first symbol of w with the last of w^R; U is the free gap u); X is the free tail v.

(c) Idea: i ≠ j + k means i > j + k or i < j + k. Every member is a^i b^j c^k in order, so split into two cases.

- **i > j + k:** a^i b^j c^k = a^k (a^j (a^m) b^j) c^k with m ≥ 1. Outer a…c pairs, inner a…b pairs, extra a's: S1 → aS1c | A1, A1 → aA1b | A2, A2 → aA2 | a.
- **i < j + k:** first pair a's with c's (S2 → aS2c). If the a's run out first, what is left is b^j c^l with j + l ≥ 1: B → bB | bC | cC, C → cC | ε. If the c's run out first, what is left is a^x b^(x+y) with y ≥ 1: D → aDb | E, E → bE | b. So S2 → aS2c | B | D.

S → S1 | S2
S1 → aS1c | A1 A1 → aA1b | A2 A2 → aA2 | a
S2 → aS2c | B | D B → bB | bC | cC C → cC | ε D → aDb | E
E → bE | b

Checked against the definition on all strings over {a, b, c} up to length 6.

(d)

```
0^n1^m0^m1^n      S → 0S1 | T,   T → 1T0 | ε
n = m + k         S → aSc | T,   T → aTb | ε          (a^k a^m b^m c^k)
n = 2m + 3k       S → aaaSc | T, T → aaTb | ε
n ≤ m ≤ 2n        S → aSb | aSbb | ε                  (each a gets one or two b's)
```

The printed condition of the last part is garbled ('0 ≥ n ≥ m ≥ 2n'); the grammar is for n ≤ m ≤ 2n.

(e) CFG = (V, Σ, R, S) as in Lesson 18. S → aSb | aAb, A → aA | a: each aSb pairs an a with a b (n ≥ 1 pairs), A adds at least one extra a, so m > n.

Practice — Lesson 19 (continued)

(f) Actual — 2007-08 Q4(a), 2009-10 Q7, 2016-17 Q8(b). (g) Actual — 2008-09 Q8(b), 2012-13 Q7(a). (h) Actual — 2009-10 Q9. (i) Actual — 2013-14 Q7(b). (j) Actual — 2010-11 Q8. (k) Actual — 2021-22 Q4(c).

(f) Grammar for palindromes over {0,1} and proof by induction; modify for {0, 1, a} and even length only; CFG for palindromes whose length is divisible by 4. (10 each)

(g) CFG for the RE $0*1(0 + 1)*$; leftmost and rightmost derivations of 00101. (8–9)

(h) CFG for equal numbers of a's and b's; leftmost derivation of aabbab. (10)

(i) CFG for {a^ib^jc^k | i, j, k ≥ 0}; parse tree for ab^2c^3. (8⅓)

(j) Using the inductive rules for REs, give a CFG generating all REs over {a, b} (Σ = {a, b, (,), ∪, ∗, ∅}). (10)

(k) A B Sloppy's CFGs for {a^ib^jc^k | i = j or j = k}: (i) S → aSb | bSc | aS | Sc | ε; (ii) S0 → S1 | S3, S1 → aS1bS2c | ε, S2 → S2c | ε, S3 → aS4bS3c | ε, S4 → aS4 | ε. Any flaws? (3 + 6)

✓ Solution

(f) P → 0P0 | 1P1 | 0 | 1 | ε. Proof that L(P) = palindromes, by induction on |w|. (⊆) every rule keeps the string equal to its reverse: if x = x^R then (0x0)^R = 0x^R0 = 0x0. (⊇) |w| ≤ 1: w ∈ {ε, 0, 1}, one rule. |w| ≥ 2 palindrome: w = axa with a ∈ {0,1} and x a shorter palindrome; by induction P ⇒∗ x, so P ⇒ aPa ⇒∗ axa. {0, 1, a}, even length only: P → 0P0 | 1P1 | aPa | ε. Length divisible by 4: S → 00S00 | 01S10 | 10S01 | 11S11 | ε (a palindrome of length 4k is uu^R with |u| = 2k; generate u two symbols at a time).

(g) S → A1B, A → 0A | ε, B → 0B | 1B | ε.

```
leftmost:  S ⇒ A1B ⇒ 0A1B ⇒ 00A1B ⇒ 001B ⇒ 0010B ⇒ 00101B ⇒ 00101
rightmost: S ⇒ A1B ⇒ A10B ⇒ A101B ⇒ A101 ⇒ 0A101 ⇒ 00A101 ⇒ 00101
```

(h) S → aSbS | bSaS | ε. Leftmost: S ⇒ aSbS ⇒ aaSbSbS ⇒ aabSbS ⇒ aabbS ⇒ aabbaSbS ⇒ aabbabS ⇒ aabbab.

(i) S → ABC, A → aA | ε, B → bB | ε, C → cC | ε. Tree for abbccc: S[A[a A[ε]] B[b B[b B[ε]]] C[c C[c C[c C[ε]]]]].

(j) Following the inductive definition (Lec05 sl. 64): R → a | b | ∅ | (R ∪ R) | (RR) | (R∗). (If ε is to be written it needs its own terminal; the given Σ has none, so ε is expressed as ∅∗.) Without full parentheses: R → R ∪ R | RR | R∗ | (R) | a | b | ∅ — correct but ambiguous.

(k) (i) Wrong: it mixes symbols out of order and ignores the counts — it generates ac (i = 1, j = 0, k = 1: neither equality holds) and acb (not even of the form a∗b∗c∗). (ii) Wrong: S1 → aS1bS2c interleaves blocks, producing aabcbc and ababcc (not a∗b∗c∗); and it cannot generate simple members such as ab, bc, abbcc, a, c (checked by program). Correct grammar: S → S1 | S2, S1 → AC, A → aAb | ε, C → cC | ε, S2 → DB, D → aD | ε, B → bBc | ε (sl. 43).

Lesson 20 — Ambiguity and Inherent Ambiguity

Slides 53–63 (Lec07)

1. Definition (sl. 54)

MEMORIZE w is derived ambiguously in G if it has two or more different parse trees — equivalently two leftmost (or two rightmost) derivations. G is ambiguous if it generates at least one string ambiguously. Ambiguity is a property of the grammar; a language may have ambiguous and unambiguous grammars. A CFL is **inherently ambiguous** if every CFG for it is ambiguous (sl. 62).

2. Expressions: Two Trees for a + a ∗ a (sl. 55–58)

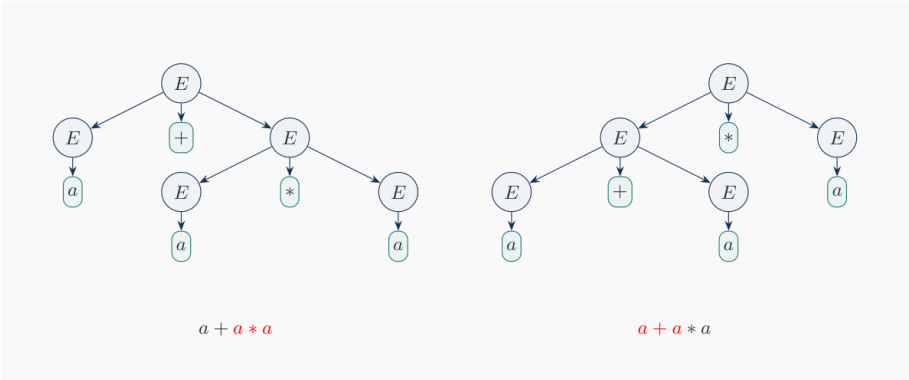


Figure 20.1 — Two parse trees for a + a ∗ a in E → E + E | E ∗ E | (E) | a (sl. 56)

The grammar treats + and ∗ alike, so it does not encode precedence. Fix with levels: F (factor) cannot be split by an operator, T (term) = product of factors, E (expression) = sum of terms: **E → E + T | T, T → T ∗ F | F, F → (E) | a** — multiplication is forced below addition and both operators associate to the left; a + a ∗ a now has one tree (checked).

3. Dangling Else and Balanced Parentheses (sl. 59–61)

S → if E then S | if E then S else S | other: 'if E then if E then other else other' has two trees (else attached to the inner or outer if). Repair: S → M | U, M → other | if E then M else M, U → if E then S | if E then M else U (every else matches the nearest unmatched if). S → SS | (S) | ε is ambiguous: ()()() groups as (()())() or ()(()()) in the tree; S → (S)S | ε is unambiguous.

4. Inherent Ambiguity (sl. 62–63)

```
L = {a^n b^n c^m d^m | n, m ≥ 1} ∪ {a^n b^m c^m d^n | n, m ≥ 1}
S → AB | C
A → aAb | ab        B → cBd | cd        (a's match b's, c's match d's)
C → aCd | aDd       D → bDc | bc        (outer a…d, inner b…c)
abcd (and every a^n b^n c^n d^n) has two trees: S⇒AB⇒abB⇒abcd  and  S⇒C⇒aDd⇒abcd
```

EXAM-READY ANSWER Why every grammar for L is ambiguous (2008-09 Q8(a), sketch from Hopcroft §5.4.4): strings a^nb^nc^nd^n belong to both parts. Any grammar must generate the first part by matching a's with b's and c's with d's in two separate, independent derivation pieces, and the second part by matching a's with d's around a b–c core (otherwise it would generate strings outside L — a pumping-lemma argument). For a^nb^nc^nd^n with n large, both kinds of matching can be applied, giving two structurally different parse trees. Hence no grammar avoids ambiguity: L is inherently ambiguous. The same argument applies to {a^nb^nc^m} ∪ {a^nb^mc^m} (2012-13 Q6(a)) with a^nb^nc^n.

Exam Approach — 'Show the Grammar Is Ambiguous'

1. Find the shortest string that both 'cases' of the grammar can produce (union grammars: a string in both parts; operator grammars: two operators; concatenation S → SS: three blocks).
2. Draw both parse trees (or two leftmost derivations) completely.
3. State: two different parse trees ⇒ the string is derived ambiguously ⇒ G is ambiguous.
4. If asked to remove ambiguity: introduce precedence levels / matched-unmatched split / force one association.

TRAP Two different derivations are NOT enough — a leftmost and a rightmost derivation of the same tree do not show ambiguity. You need two different trees, or two different LEFTMOST derivations.

Practice — Lesson 20

(a) Actual — 2022-23 Q6, Jan-2020 Q6, 2008-09 Q8(a). (b) Actual — 2023-24 Q5. (c) Actual — 2021-22 Q6, 2019-20 Q6. (d) Actual — 2017-18 Q6, 2012-13 Q5(a), 2006-07 Q7. (e) Actual — 2008-09 Q8(c). (f) Actual — 2014-15 Q6(a). (g) Actual — 2021-22 Q4(d). (h) Actual — 2007-08 Q3(b), 2011-12 Q8(a).

(a) Find a CFG for L = {a^nb^nc^md^m | n, m ≥ 1} ∪ {a^nb^mc^md^n | n, m ≥ 1} and show it is ambiguous by giving two parse trees. (10 + 5)

(b) Design a CFG for {a^ib^jc^k | i = j or i = k; i, j, k ≥ 1} and show it is ambiguous. (10 + 5)

(c) Define ambiguous grammar and inherently ambiguous language. Is S → AB | C, A → aAb | ab, B → cB | c, C → aCc | aDc, D → bD | b ambiguous? (6 + 9)

(d) E → E + E | E ∗ E | (E) | a (with I → a | b | Ia | Ib | I0 | I1 in 2006-07): show it is ambiguous; remove the ambiguity. (7 + 8)

(e) S → aS | aSbS | ε: show aab has two parse trees, two leftmost and two rightmost derivations. (5⅓)

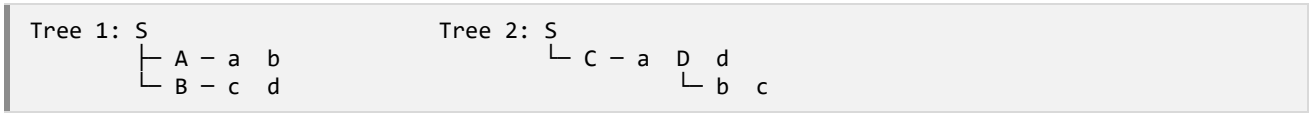
(f) Explain why S → 0A | 1B, A → 0AA | 1S | 1, B → 1BB | 0S | 0 is ambiguous. (7)

(g) A grammar for an infinite language; string s. Comment on the ambiguity of s and of the grammar if: (i) only one parse tree for s but more than one for other strings; (ii) one parse tree for s and for every other string; (iii) more than one leftmost derivation for s. (10)

(h) What is ambiguity in grammars? How do we remove it? What is inherent ambiguity? (8)

✓ Solution

(a) Grammar and two trees for abcd in §4:



(b) i = j part: S1 → XC, X → aXb | ab, C → cC | c. i = k part: S2 → aS2c | aBc, B → bB | b. S → S1 | S2. abc is in both parts: S ⇒ S1 ⇒ XC ⇒ abC ⇒ abc and S ⇒ S2 ⇒ aBc ⇒ abc — two parse trees (checked).

S → XC | Y X → aXb | ab C → cC | c
Y → aYc | aBc B → bB | b

(c) Definitions: §1. L(AB) = {a^nb^nc^m | n, m ≥ 1}; L(C) = {a^nb^mc^n | n, m ≥ 1}. The string abc lies in both: S ⇒ AB ⇒ abB ⇒ abc and S ⇒ C ⇒ aDc ⇒ abc — two different leftmost derivations (two trees), so the grammar is ambiguous (verified: abc has exactly 2 trees). The language {a^nb^nc^m} ∪ {a^nb^mc^n} is in fact inherently ambiguous (strings a^nb^nc^n).

(d) a + a ∗ a: tree 1 E ⇒ E + E with the right E ⇒ E ∗ E (means a + (a∗a)); tree 2 E ⇒ E ∗ E with the left E ⇒ E + E (means (a+a)∗a) — Figure 20.1. (Even a + a + a has two trees.) Unambiguous grammar: E → E + T | T, T → T ∗ F | F, F → (E) | I, I → a | b | Ia | Ib | I0 | I1.

(e)

```
Tree 1: S → aS, that S → aSbS, inner S's → ε        Tree 2: S → aSbS, first S → aS, inner S's → ε
LM 1: S ⇒ aS ⇒ aaSbS ⇒ aabS ⇒ aab                    LM 2: S ⇒ aSbS ⇒ aaSbS ⇒ aabS ⇒ aab
RM 1: S ⇒ aS ⇒ aaSbS ⇒ aaSb ⇒ aab                    RM 2: S ⇒ aSbS ⇒ aSb ⇒ aaSb ⇒ aab
```

(f) The grammar generates equal numbers of 0s and 1s but gives some strings two trees. The shortest is 001011 (found by counting trees): S ⇒ 0A ⇒ 00AA ⇒ 001A ⇒ 0010AA ⇒ 00101A ⇒ 001011 (first A → 1, second A → 0AA) versus S ⇒ 0A ⇒ 00AA ⇒ 001SA ⇒ 0010AA ⇒ 00101A ⇒ 001011 (first A → 1S, S → 0A) — two leftmost derivations, two trees.

(g) (i) s itself is unambiguous, but the grammar IS ambiguous (one ambiguous string suffices). (ii) Neither s nor the grammar is ambiguous. (iii) s is ambiguous (two leftmost derivations = two trees), hence the grammar is ambiguous; the language may still have some other unambiguous grammar unless it is inherently ambiguous.

(h) Definition: §1. Removal: rewrite the grammar so that each string has one structure — precedence levels (E/T/F), associativity (left-recursive E → E + T), separating matched/unmatched statements (dangling else), S → (S)S instead of S → SS | (S). There is no general algorithm (ambiguity is undecidable, Atif Lec 7 sl. 77), and for inherently ambiguous languages it is impossible.

Lesson 21 — The Pumping Lemma for Context-Free Languages

Slides 64–97 (Lec07) · also Atif Lec 3 sl. 123–129

1. Idea and Statement (sl. 65–70)

A long string needs a tall parse tree; with finitely many variables some variable A repeats on a root-to-leaf path: A ⇒∗ vAy and A ⇒∗ x, so A ⇒∗ v^iAy^i ⇒∗ v^ixy^i — two pieces are pumped together.

MEMORIZE If L is context-free there is p ≥ 1 such that every w ∈ L with |w| ≥ p can be written w = uvxyz with (1) |vxy| ≤ p, (2) |vy| > 0, (3) uv^ixy^iz ∈ L for every i ≥ 0. To prove non-CF: ∀p ∃w ∀ valid uvxyz ∃i: uv^ixy^iz ∉ L.

2. Standard Proofs (sl. 71–89)

Language	Witness	Key fact	Pump
a^nb^nc^n	a^pb^pc^p	vxy touches at most 2 adjacent blocks	i = 2: order broken or one block unchanged
a^nb^mc^nd^m	a^pb^pc^pd^p	matching blocks are not adjacent	i = 2: #a ≠ #c or #b ≠ #d
a^q, q prime	a^q, q > p prime	d = |vy|, 1 ≤ d < q	i = q + 1: length q(1 + d)
a^nb^(n²)	a^pb^(p²)	β = 2pα + α² impossible	i = 2

TRAP Pump BOTH v and y; consider every placement of vxy (inside a block, across a boundary); do not choose v and y yourself (sl. 91–93).

Exam Approach

1. Witness with every block of length p (or a multiple).
2. Use |vxy| ≤ p: vxy lies within two adjacent blocks.
3. Case 1: v or y contains two different symbols → pumping breaks the order. Case 2: each in one block → pumping changes at most two block lengths and the third is fixed.
4. Conclude with the contradiction.

Practice — Lesson 21

(a) Actual — 2013-14 Q7(c). (b) Constructed (slides' practice list, sl. 96).

(a) Show that L = {a^nb^(2n)c^(3n) | n ≥ 0} is not context-free. (6)

(b) Show that {0^n1^n2^n} and {a^nb^(n³)} are not context-free.

✓ Solution

(a) Assume L is CF with pumping length p; w = a^pb^(2p)c^(3p) ∈ L, |w| ≥ p. Let w = uvxyz, |vxy| ≤ p, |vy| > 0. Since |vxy| ≤ p, vxy cannot contain both a's and c's (the b-block of length 2p lies between). Case 1: v or y contains two different symbols — then uv²xy²z has symbols out of order (e.g. …ab…ab…) ∉ L. Case 2: v and y each lie inside one block. Then pumping with i = 2 changes the lengths of at most two of the three blocks; if it changes #a, the c-block (length 3p) is unchanged, so #c ≠ 3·#a; if it changes no a's, then #a = p but #b or #c changes, so #b ≠ 2p or #c ≠ 3p. In every case uv²xy²z ∉ L — contradiction. L is not context-free.

(b) 0^n1^n2^n: exactly as (a) with blocks p, p, p (sl. 73–77). a^nb^(n³): w = a^pb^(p³); with α = #a(vy), β = #b(vy) and i = 2 we need p³ + β = (p + α)³; if α = 0 the b-count changes alone; if α ≥ 1 the right side grows by ≥ 3p² > p ≥ β. Contradiction.

Quick Notes — CFG

Revision points for Lessons 18–21.

L18 · Basics

- G = (V, Σ, R, S); A → α; context-free = replace A regardless of neighbours.
- Leftmost/rightmost fix the order; tree ↔ one lm ↔ one rm derivation.
- Tree: root S, internal variables, leaves terminals/ε, node+children = rule; yield = leaves.
- Recursive inference = bottom-up; derivation = top-down.

L19 · Design

- Balanced S → (S)S | ε. #0=#1: S → 0S1S | 1S0S | ε. ≥k 1s: X1X1…X.
- Palindromes P → 0P0|1P1|0|1|ε. a^nb^mc^md^n: S → aSd | T, T → bTc | ε.
- i=j or j=k: S1 | S2 with A→aAb, C→cC / D→aD, B→bBc. i ≠ j+k: two cases (S1 >, S2 <).
- n ≤ m ≤ 2n: S → aSb | aSbb | ε. Regular ⊂ CFL (R_i → aR_j).

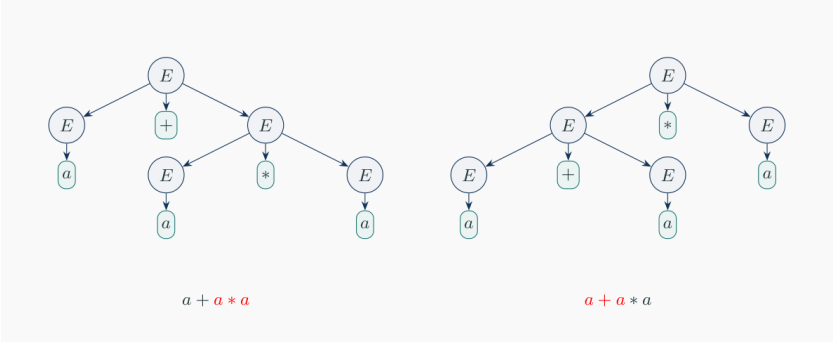
L20 · Ambiguity

- Ambiguous = some string has 2 trees / 2 leftmost derivations.
- E+E|E∗E: a+a∗a two trees; fix E → E+T | T, T → T∗F | F, F → (E) | a.
- Union grammars: string in both parts (abc, abcd) ⇒ two trees.
- Inherently ambiguous: a^nb^nc^md^m ∪ a^nb^mc^md^n (strings a^nb^nc^nd^n).

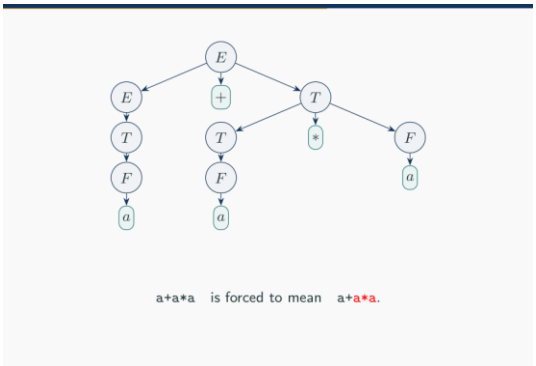
L21 · CFL pumping

- uvxyz, |vxy| ≤ p, |vy| > 0, pump both. vxy spans ≤ 2 adjacent blocks.
- a^nb^nc^n, a^nb^mc^nd^m, primes, a^nb^(n²): not CF.

Key Diagrams to Reproduce



Two parse trees for a + a ∗ a (sl. 56)



Unambiguous E/T/F tree (sl. 58)